UAA7 – Bases de données

SQL DML

Exercices supplémentaires

Ces exercices complètent la section 5.2 du cours (Data Manipulation Language). Ils travaillent :

- l'ajout de lignes (`INSERT INTO ... VALUES`), avec la méthode implicite et la méthode explicite ;
- les colonnes omises : valeur `NULL` ou valeur par défaut (`DEFAULT`) ;
- la modification de lignes (`UPDATE ... SET ... WHERE`), y compris avec un calcul (`prix * 1.05`) ;
- la suppression de lignes (`DELETE FROM ... WHERE`) ;
- les conditions de sélection : =, <, `BETWEEN`, `IN`, `LIKE`, `IS NULL` ;
- les violations des contraintes d'intégrité : `PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `FOREIGN KEY`, `CHECK` ;
- les effets d'une suppression sur les tables liées : `ON DELETE RESTRICT`, `CASCADE` et `SET NULL` (section 5.1).

Mode d'emploi. Tous les exercices portent sur la base de données d'un club de sport, présentée à la page suivante. Son script, `club_sport.sql`, crée les tables et insère les données de départ. Télécharge-le sur la page du cours :

> https://lucasembrechts.github.io/UAA7/club_sport.sql

Copie tout son contenu dans DB Disco et exécute-le avant de commencer. Quand un exercice indique « on repart des données initiales », exécute à nouveau le script complet : il supprime les tables et les recrée avec leurs données de départ.

La base de données du club

Le club propose plusieurs sports. Chaque membre peut pratiquer un sport (ou aucun pour l'instant). Chaque sport a ses séances hebdomadaires. Le club prête aussi des casiers à certains membres.

Voici les données de départ :

SPORT

code	nom	cotisation
NAT	Natation	180.00
TEN	Tennis	250.00
BAD	Badminton	120.00
ESC	Escalade	300.00

MEMBRE

id	nom	prenom	email	dateNaissance	dateInscription	sport
1	Dubois	Léa	lea.dubois@mail.be	2008-04-12	2024-09-02	NAT
2	Martin	Hugo	hugo.martin@mail.be	2007-11-30	2023-09-05	TEN
3	Lambert	Inès	NULL	2009-02-03	2025-01-15	NAT
4	Renard	Tom	tom.renard@mail.be	2008-07-21	2024-09-02	ESC
5	Petit	Sarah	sarah.petit@mail.be	2006-12-08	2022-10-10	NULL
6	Rousseau	Noah	noah.rousseau@mail.be	2009-05-17	2025-03-01	BAD

SEANCE

id	sport	jour	heure	places
1	NAT	lundi	17:00	20
2	NAT	jeudi	18:00	20
3	TEN	mercredi	14:00	8
4	ESC	samedi	10:00	12
5	BAD	mardi	16:30	16

CASIER

numero	membre
101	1
102	2
103	NULL
104	4

Exercice 1 – Ajouter des lignes

On part des données initiales. Écris les instructions `INSERT` qui enregistrent les informations suivantes. Pour chacune, choisis la méthode (implicite ou explicite) la plus adaptée.

a) Le club ouvre une section Volley-ball : code `VOL`, cotisation de 150 €.

b) Emma Leclercq, née le 3 juin 2009, s'inscrit aujourd'hui au volley-ball. Elle n'a pas d'adresse e-mail. Elle reçoit le numéro de membre 7.

c) Une séance de volley-ball (numéro 6) a lieu le vendredi à 17 h 30. Elle compte 14 places.

d) Le casier 105 est attribué à Emma.

e) Lucas Bernard (membre numéro 8, né le 14 février 2008, adresse lucas.bernard@mail.be) s'est inscrit le 1er septembre 2025, mais il n'a pas encore choisi de sport.

Réponds ensuite aux questions suivantes :

f) Dans quel ordre faut-il exécuter les instructions a, b, c et d ? Que se passe-t-il si on exécute b avant a ?

g) Dans l'instruction b, tu n'as pas donné de valeur aux colonnes `email` et `dateInscription`. Quelle valeur reçoit chacune d'elles ? Pourquoi ces deux valeurs sont-elles différentes ?

h) Pourquoi, en e, faut-il donner la date d'inscription alors qu'on ne l'a pas fait en b ?

Exercice 2 – Accepté ou refusé ?

On part des données initiales. **Chaque instruction est exécutée seule**, à partir des données initiales : on ne tient pas compte des instructions qui la précèdent. Le commentaire indique ce qu'on veut faire.

```
-- a) Ajouter le volley-ball
INSERT INTO SPORT VALUES ('VOL', 'Volley-ball', 150);
-- b) Ajouter la natation synchronisée
INSERT INTO SPORT VALUES ('NAT', 'Natation synchronisée', 200);
-- c) Ajouter le judo (cotisation pas encore fixée)
INSERT INTO SPORT (code, nom) VALUES ('JUD', 'Judo');
-- d) Ajouter la danse
INSERT INTO SPORT VALUES ('DAN', 'Danse', -20);
-- e) Inscrire Zoé Lefèvre au badminton
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance, sport)
VALUES (7, 'Lefèvre', 'Zoé', '2010-01-25', 'BAD');
-- f) Inscrire Max Leroy au tennis
INSERT INTO MEMBRE (id, nom, prenom, email, dateNaissance, sport)
VALUES (8, 'Leroy', 'Max', 'hugo.martin@mail.be', '2008-03-03', 'TEN');
-- g) Inscrire Jade Fontaine au football
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance, sport)
VALUES (9, 'Fontaine', 'Jade', '2009-10-10', 'FOO');
-- h) Inscrire Eva Simon
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance, dateInscription)
VALUES (10, 'Simon', 'Eva', '2025-06-01', '2024-09-01');
-- i) Inscrire Julie Moreau (prénom Julie, nom Moreau) à la natation
INSERT INTO MEMBRE
VALUES (11, 'Julie', 'Moreau', 'julie.moreau@mail.be',
        '2009-09-09', '2025-09-01', 'NAT');
-- j) Inscrire Luis Garcia
INSERT INTO MEMBRE (id, nom, prenom, email, dateNaissance)
VALUES (12, 'Garcia', 'Luis', 'luis.garcia.mail.be', '2008-08-08');
-- k) Inscrire Léon Blanc
INSERT INTO MEMBRE (nom, prenom, dateNaissance)
VALUES ('Blanc', 'Léon', '2009-01-01');
-- l) Ajouter une séance de badminton le samedi
INSERT INTO SEANCE VALUES (7, 'BAD', 'Samedi', '11:00', 12);
-- m) Prêter le casier 105 à Inès Lambert
INSERT INTO CASIER VALUES (105, 3);
-- n) Prêter le casier 106 à Léa Dubois
INSERT INTO CASIER VALUES (106, 1);
-- o) Inscrire Jade Fontaine au badminton
INSERT INTO MEMBRE VALUES (13, 'Fontaine', 'Jade', NULL, '2009-10-10', 'BAD');
```

Pour chaque instruction, complète le tableau. Si elle est refusée, indique quelle contrainte d'intégrité est violée et corrige l'instruction pour qu'elle soit acceptée. Attention : une instruction peut être acceptée et pourtant ne pas faire ce qu'on voulait.

	Acceptée ?	Contrainte violée / problème	Correction
a			
b			
c			
d			
e			
f			
g			
h			
i			
j			
k			
l			
m			
n			
o			

Exercice 3 – Modifier des lignes

On repart des données initiales. Écris les instructions `UPDATE` suivantes et exécute-les **dans l'ordre**. Après chaque instruction, note le nombre de lignes modifiées.

a) La cotisation de l'escalade augmente de 10 %.

b) Inès Lambert (membre numéro 3) communique son adresse e-mail : ines.lambert@mail.be.

c) Les membres qui n'ont pas encore de sport sont inscrits au badminton.

d) Toutes les séances de natation passent à 25 places.

e) La séance de tennis du mercredi est déplacée à 15 h 00 et compte désormais 10 places (une seule instruction).

f) Les sports dont la cotisation est comprise entre 100 € et 200 € (bornes comprises) baissent de 15 €.

g) Les séances du samedi et du dimanche commencent désormais à 9 h 30.

h) Les noms de famille de tous les membres sont enregistrés en majuscules (utilise la fonction `UPPER`).

i) On veut faire passer la séance numéro 2 à 35 places. Écris l'instruction et exécute-la. Que se passe-t-il ? Pourquoi ?

j) Que se serait-il passé si tu avais oublié la clause `WHERE` en a ?

k) Quel est le contenu final de la table SPORT ?

Exercice 4 – Supprimer des lignes

On repart des données initiales. Rappel des comportements à la suppression :

Clé étrangère	Fait référence à	À la suppression de la ligne référencée
`MEMBRE.sport`	`SPORT.code`	`ON DELETE RESTRICT`
`SEANCE.sport`	`SPORT.code`	`ON DELETE CASCADE`
`CASIER.membre`	`MEMBRE.id`	`ON DELETE SET NULL`

Pour chaque demande, exécutée **dans l'ordre** :

- **prévois** combien de lignes seront supprimées, dans quelles tables, et ce qui arrive aux tables liées ;
- **écris** la requête, puis exécute-la et compare avec ta prévision.

a) Supprimer la séance du mardi.

b) Supprimer les membres dont l'adresse e-mail est inconnue.

c) Supprimer le membre Tom Renard (numéro 4). Que devient le casier 104 ?

d) Supprimer le sport Tennis. Que se passe-t-il ?

e) Que faut-il faire pour pouvoir supprimer le tennis tout en gardant Hugo Martin comme membre du club ? Fais-le. Que devient la séance de tennis du mercredi ?

f) Supprimer les membres dont le nom commence par « R ». Combien de lignes sont supprimées ? Pourquoi pas deux ?

Exercice 5 – Chasse aux erreurs

Un élève a écrit le script ci-dessous. Le commentaire au-dessus de chaque instruction indique ce qu'il voulait faire. On suppose que chaque instruction est exécutée à partir des données initiales, comme si les instructions précédentes avaient été corrigées.

Le script contient **8 erreurs**. Certaines empêchent son exécution. D'autres passent sans message mais ne font pas ce qui était voulu.

```
-- 1. Ajouter le yoga (cotisation : 90 €)
INSERT INTO SPORT VALUES ('YOG', "Yoga", 90);

-- 2. Ajouter Paul Durand (numéro 20), né le 4 avril 2009
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance)
VALUES (20, 'Durand', 'Paul');

-- 3. Ajouter Lina Roux (numéro 21), née le 25 décembre 2009,
--    inscrite au yoga
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance, sport)
VALUES (21, 'Roux', 'Lina', '2009-25-12', 'YOG');

-- 4. Le yoga coûte finalement 100 € et s'appelle « Yoga doux »
UPDATE SPORT SET cotisation = 100 AND nom = 'Yoga doux'
WHERE code = 'YOG';

-- 5. Inscrire au badminton les membres qui n'ont pas de sport
UPDATE MEMBRE SET sport = 'BAD' WHERE sport = NULL;

-- 6. Ajouter 5 places à la séance du jeudi
UPDATE SEANCE SET places = places + 5;

-- 7. Supprimer la séance du lundi
DELETE * FROM SEANCE WHERE jour = 'lundi';

-- 8. Supprimer les membres dont le nom commence par D
DELETE FROM MEMBRE WHERE nom LIKE 'D*';
```

a) Repère les 8 erreurs. Pour chacune, indique si l'instruction est refusée ou si elle passe sans message, et explique l'erreur en une phrase.

b) Réécris le script corrigé.

Exercice 6 – Que contient la table ?

On repart des données initiales. Les instructions suivantes sont exécutées **dans l'ordre**, sans les corriger.

```
-- 1
INSERT INTO SPORT VALUES ('VOL', 'Volley-ball', 150);
-- 2
UPDATE SPORT SET cotisation = cotisation + 20 WHERE cotisation < 200;
-- 3
UPDATE SPORT SET nom = UPPER(nom) WHERE code IN ('NAT', 'VOL');
-- 4
DELETE FROM SPORT WHERE cotisation > 260;
-- 5
INSERT INTO SPORT (code, nom, cotisation) VALUES ('bad', 'Badminton', 140);
-- 6
DELETE FROM SPORT WHERE nom LIKE 'V%';
-- 7
UPDATE SPORT SET cotisation = cotisation * 0.9 WHERE code = 'TEN';
```

a) Sans exécuter le script, complète le tableau. Pense aux contraintes de toutes les tables, pas seulement à celles de SPORT.

Instruction	Acceptée ?	Nombre de lignes touchées	Explication
1			
2			
3			
4			
5			
6			
7			

b) Donne le contenu final de la table SPORT. Exécute ensuite le script pour vérifier.

Exercice 7 – Questions de réflexion

a) Quelle est la différence entre `DELETE FROM MEMBRE;` et `DROP TABLE MEMBRE;` ? Laquelle de ces deux instructions est acceptée sur la base du club ? Pourquoi l'autre est-elle refusée ?

b) Pourquoi la méthode explicite est-elle plus sûre que la méthode implicite ? Appuie-toi sur les instructions i et o de l'exercice 2.

c) Pourquoi la condition `WHERE email = NULL` ne trouve-t-elle jamais aucune ligne ? Que faut-il écrire ?

d) Une instruction `UPDATE` peut-elle violer une contrainte d'intégrité ? Donne deux exemples sur la base du club, avec deux contraintes différentes.

e) L'instruction `UPDATE SEANCE SET places = places + 12;` est refusée, alors que seules les deux séances de natation dépasseraient 30 places. Les trois autres séances sont-elles modifiées ? Pourquoi ?

f) Quel réflexe faut-il avoir avant d'exécuter un `UPDATE` ou un `DELETE` ?

Corrigé

Toutes les requêtes du corrigé ont été testées sur PostgreSQL (le SGBD de DB Disco).

Exercice 1

```
-- a
INSERT INTO SPORT VALUES ('VOL', 'Volley-ball', 150.00);
-- b (explicite : on ne donne pas toutes les colonnes)
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance, sport)
VALUES (7, 'Leclercq', 'Emma', '2009-06-03', 'VOL');
-- c (implicite : toutes les colonnes sont connues)
INSERT INTO SEANCE VALUES (6, 'VOL', 'vendredi', '17:30', 14);
-- d
INSERT INTO CASIER VALUES (105, 7);
-- e
INSERT INTO MEMBRE (id, nom, prenom, email, dateNaissance, dateInscription)
VALUES (8, 'Bernard', 'Lucas', 'lucas.bernard@mail.be', '2008-02-14', '2025-09-
01');
```

f) Il faut créer une ligne avant de pouvoir y faire référence : d'abord le sport (a), ensuite Emma (b) et la séance (c), qui font référence au sport, enfin le casier (d), qui fait référence à Emma. Si on exécute b avant a, l'insertion est refusée : la clé étrangère `sport` vaudrait `VOL`, un code qui n'existe pas encore dans SPORT.

g) `email` reçoit `NULL`, car aucune valeur par défaut n'est déclarée. `dateInscription` reçoit la date du jour, car la colonne est déclarée avec `DEFAULT CURRENT_DATE`.

h) Lucas s'est inscrit le 1er septembre 2025, pas aujourd'hui. Si on omet la colonne, la valeur par défaut (la date du jour) serait enregistrée, ce qui serait faux. Le sport est omis : il vaut `NULL`.

Exercice 2

	Acceptée ?	Contrainte violée / problème	Correction
a	Oui	–	–
b	Non	`PRIMARY KEY` : le code NAT existe déjà.	Choisir un autre code, par exemple `'NSY'`.
c	Non	`NOT NULL` : la cotisation est obligatoire et n'a pas de valeur par défaut.	Donner une cotisation : `VALUES ('JUD', 'Judo', 160)`.
d	Non	`CHECK (cotisation >= 0)` : la cotisation est négative.	Mettre une cotisation positive, par exemple `20`.
e	Oui	–	`email` vaut `NULL` et `dateInscription` la date du jour (`DEFAULT`). Le `CHECK` sur l'e-mail n'est pas violé : avec `NULL`, la condition n'est pas fausse, elle est inconnue, et le SGBD l'accepte.
f	Non	`UNIQUE` sur `email` : l'adresse est déjà celle de Hugo Martin.	Donner l'adresse de Max, par exemple `'max.leroy@mail.be'`.
g	Non	`FOREIGN KEY` : le sport FOO n'existe pas dans SPORT.	Créer d'abord le sport, ou utiliser un code existant.
h	Non	`CHECK (dateInscription > dateNaissance)` : Eva serait inscrite avant d'être née (les dates sont inversées).	Remettre les dates dans le bon ordre.

	Acceptée ?	Contrainte violée / problème	Correction
i	Oui, mais…	Aucune contrainte violée, mais avec la méthode implicite, les valeurs suivent l'ordre des colonnes : « Julie » est enregistré comme nom et « Moreau » comme prénom.	Inverser les deux valeurs, ou utiliser la méthode explicite.
j	Non	`CHECK (email LIKE '%@%')` : l'adresse ne contient pas de @.	`'luis.garcia@mail.be'`
k	Non	`PRIMARY KEY` : `id` est omis, il vaudrait `NULL`. Une clé primaire ne peut pas être `NULL`.	Donner un numéro, par exemple 14.
l	Non	`CHECK` sur `jour` : « Samedi » avec une majuscule ne fait pas partie de la liste.	Écrire `'samedi'`.
m	Oui	–	Inès n'avait pas de casier : le casier 105 lui est attribué.
n	Non	`UNIQUE` sur `membre` : Léa a déjà le casier 101.	Prêter le casier à un autre membre, ou libérer d'abord le 101.
o	Non	Méthode implicite avec une valeur manquante : les 6 valeurs remplissent les 6 premières colonnes dans l'ordre, donc `'BAD'` arrive dans `dateInscription` (type `DATE`) et provoque une erreur de type.	Utiliser la méthode explicite, ou ajouter la date d'inscription avant `'BAD'`.

Exercice 3

```
-- a  (1 ligne : 300.00 devient 330.00)
UPDATE SPORT SET cotisation = cotisation * 1.10 WHERE code = 'ESC';
-- b  (1 ligne)
UPDATE MEMBRE SET email = 'ines.lambert@mail.be' WHERE id = 3;
-- c  (1 ligne : Sarah Petit)
UPDATE MEMBRE SET sport = 'BAD' WHERE sport IS NULL;
-- d  (2 lignes : séances 1 et 2)
UPDATE SEANCE SET places = 25 WHERE sport = 'NAT';
-- e  (1 ligne)
UPDATE SEANCE SET heure = '15:00', places = 10
WHERE sport = 'TEN' AND jour = 'mercredi';
-- f  (2 lignes : natation 180 -> 165, badminton 120 -> 105)
UPDATE SPORT SET cotisation = cotisation - 15 WHERE cotisation BETWEEN 100 AND
200;
-- g  (1 ligne : séance 4 ; aucune séance le dimanche)
UPDATE SEANCE SET heure = '09:30' WHERE jour IN ('samedi', 'dimanche');
-- h  (6 lignes : pas de WHERE, c'est voulu)
UPDATE MEMBRE SET nom = UPPER(nom);
-- i  (refusée)
UPDATE SEANCE SET places = 35 WHERE id = 2;
```

i) L'instruction est refusée : elle viole la contrainte `CHECK (places BETWEEN 1 AND 30)`. Un `UPDATE` doit respecter les mêmes contraintes qu'un `INSERT`.

j) Sans `WHERE`, toutes les lignes sont modifiées : les quatre cotisations auraient augmenté de 10 %.

k) Contenu final de SPORT. L'escalade (330.00) n'est plus dans l'intervalle en f, car elle a été augmentée en a.

code	nom	cotisation
NAT	Natation	165.00
TEN	Tennis	250.00
BAD	Badminton	105.00
ESC	Escalade	330.00

Exercice 4

	Requête	Effet
a	`DELETE FROM SEANCE WHERE jour = 'mardi';`	1 ligne supprimée dans SEANCE (séance 5). Aucune table ne fait référence à SEANCE.
b	`DELETE FROM MEMBRE WHERE email IS NULL;`	1 ligne supprimée dans MEMBRE (Inès Lambert). Elle n'avait pas de casier : aucun autre effet.
c	`DELETE FROM MEMBRE WHERE id = 4;`	1 ligne supprimée dans MEMBRE. Le casier 104 n'est pas supprimé : sa colonne `membre` devient `NULL` (`ON DELETE SET NULL`). Le casier est libéré.
d	`DELETE FROM SPORT WHERE code = 'TEN';`	Refusée : Hugo Martin pratique encore le tennis (`ON DELETE RESTRICT`). Rien n'est supprimé.
e	`UPDATE MEMBRE SET sport = NULL WHERE sport = 'TEN';` `DELETE FROM SPORT WHERE code = 'TEN';`	On retire d'abord le tennis à Hugo (1 ligne modifiée), puis on supprime le sport (1 ligne). La séance 3 du mercredi est supprimée automatiquement (`ON DELETE CASCADE`).
f	`DELETE FROM MEMBRE WHERE nom LIKE 'R%';`	1 seule ligne supprimée (Noah Rousseau) : Tom Renard a déjà été supprimé en c.

À la fin, il reste trois membres : Léa Dubois (natation), Hugo Martin et Sarah Petit (sans sport).

Exercice 5

	Refusée ou silencieuse ?	Erreur
1	Refusée	`"Yoga"` entre guillemets doubles désigne un nom de colonne (« column "Yoga" does not exist »). Les textes s'écrivent entre apostrophes : `'Yoga'`.
2	Refusée	4 colonnes annoncées pour 3 valeurs : il manque la date de naissance.
3	Refusée	Les dates s'écrivent `'AAAA-MM-JJ'`. Ici, le mois vaudrait 25. Il faut `'2009-12-25'`.
4	Refusée	Les colonnes modifiées se séparent par une virgule, pas par `AND`. PostgreSQL refuse l'instruction (d'autres SGBD l'acceptent et enregistrent une valeur absurde).
5	Silencieuse	`= NULL` n'est jamais vrai : aucune ligne n'est modifiée (UPDATE 0). Il faut `IS NULL`.
6	Silencieuse	Le `WHERE` est oublié : les 5 séances reçoivent 5 places de plus (UPDATE 5).
7	Refusée	Erreur de syntaxe : `DELETE` supprime des lignes entières, on ne choisit pas de colonnes. Pas de `*`.
8	Silencieuse	Le joker de `LIKE` est `%`, pas `*` : aucune ligne n'est supprimée (DELETE 0).

Script corrigé :

```
INSERT INTO SPORT VALUES ('YOG', 'Yoga', 90);
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance)
VALUES (20, 'Durand', 'Paul', '2009-04-04');
INSERT INTO MEMBRE (id, nom, prenom, dateNaissance, sport)
VALUES (21, 'Roux', 'Lina', '2009-12-25', 'YOG');
UPDATE SPORT SET cotisation = 100, nom = 'Yoga doux' WHERE code = 'YOG';
UPDATE MEMBRE SET sport = 'BAD' WHERE sport IS NULL;
UPDATE SEANCE SET places = places + 5 WHERE jour = 'jeudi';
DELETE FROM SEANCE WHERE jour = 'lundi';
DELETE FROM MEMBRE WHERE nom LIKE 'D%';
```

Exercice 6

Instruction	Acceptée ?	Lignes touchées	Explication
1	Oui	1	Le volley-ball est ajouté.
2	Oui	3	Natation 180 → 200, badminton 120 → 140, volley-ball 150 → 170.
3	Oui	2	NATATION et VOLLEY-BALL.
4	Non	0	Seule l'escalade (300) dépasse 260, mais Tom Renard la pratique (`ON DELETE RESTRICT`). Toute l'instruction est refusée.
5	Non	0	`'bad'` est différent de `'BAD'` : la clé primaire est respectée. Mais le nom Badminton existe déjà (`UNIQUE`).
6	Oui	1	VOLLEY-BALL commence par V. Aucun membre ni séance ne pratique le volley-ball.
7	Oui	1	Tennis : 250 × 0,9 = 225.00.

Contenu final de SPORT :

code	nom	cotisation
NAT	NATATION	200.00
TEN	Tennis	225.00
BAD	Badminton	140.00
ESC	Escalade	300.00

Exercice 7

a) `DELETE FROM MEMBRE;` supprime toutes les lignes, mais la table reste (colonnes et contraintes) : on peut y insérer de nouveaux membres. `DROP TABLE MEMBRE;` supprime la table elle-même. Ici, le `DELETE` est accepté (les casiers sont libérés grâce à `ON DELETE SET NULL`). Le `DROP TABLE` est refusé, car la table CASIER fait référence à MEMBRE par une clé étrangère.

b) Avec la méthode explicite, on voit quelle valeur va dans quelle colonne. La méthode implicite dépend de l'ordre de déclaration des colonnes : une inversion passe inaperçue (instruction i) et une valeur oubliée décale toutes les suivantes (instruction o). La méthode explicite permet aussi d'omettre les colonnes facultatives ou qui ont une valeur par défaut.

c) `NULL` représente une valeur inconnue. Comparer une valeur inconnue avec = ne donne jamais « vrai », donc aucune ligne n'est sélectionnée. Il faut écrire `WHERE email IS NULL` (ou `IS NOT NULL`).

d) Oui. Exemples : `UPDATE SEANCE SET places = 35 WHERE id = 2;` viole le `CHECK` sur les places. `UPDATE SPORT SET code = 'NTN' WHERE code = 'NAT';` est refusé, car des membres font référence au code NAT (`FOREIGN KEY`). On peut aussi citer `UNIQUE` (donner à un membre l'adresse e-mail d'un autre) ou `NOT NULL` (`SET nom = NULL`).

e) Non : aucune séance n'est modifiée. Une instruction est exécutée en entier ou pas du tout. Si une seule ligne viole une contrainte, toute l'instruction est annulée et la base reste dans l'état où elle était.

f) Écrire et relire d'abord la condition `WHERE`, en se demandant quelles lignes elle désigne. Un `UPDATE` ou un `DELETE` sans `WHERE` touche toutes les lignes, et il n'y a pas de bouton « annuler ». Quand l'extraction de données sera vue (section 5.3), on pourra tester la condition avec un `SELECT` avant de modifier ou de supprimer.